



Rapport de fin de projet

Projet informatique individuel - ENSC Bordeaux inp

Client : Bâtiment SMART - Université de Bordeaux

Auteur : TONON Thiméo

Table des matières

Résumé	2
Contexte du projet	2
Objectifs et choix techniques	2
Déroulement du projet	4
Architecture de l'application	5
Back-end	5
Front-end	7
Description des éléments du projet	9
Authentification	9
Équipements	10
Relations entre équipements	10
Labels	10
Exemplaires	11
Disponibilités	11
Réservations, sessions et emprunts	12
Poursuite du projet	13
Bilan personnel	13
Conclusion	14
Annexe	15

Résumé

Ce rapport rassemble les détails des travaux effectués durant la réalisation du projet SMART Réserve, une application de réservation d'équipements dédiée à l'établissement SMART (Université de Bordeaux). Ce rapport contient le contexte et le déroulement du projet, l'architecture et les spécifications techniques des éléments constitutifs du projet.

Contexte du projet

Le SMART est un centre sportif d'analyse du mouvement et de la performance situé à PESSAC. Ce centre héberge de nombreuses technologies d'analyse et de visualisation du mouvement et de la performance, et nécessite d'avoir une plateforme pour gérer toute la logistique des équipements inclus.

Cependant, la gestion des différentes demandes de réservation du matériel des nombreux utilisateurs du SMART est peu évidente en raison d'un manque d'une plateforme dédiée.

L'objectif de ce projet est donc de créer une plateforme web assurant la gestion logistique des équipements et des réservations. Elle permettra d'assurer la fluidité des échanges entre les étudiants, chercheurs et clients qui viennent réserver des équipements au SMART, et de faciliter le traitement des demandes de réservation.

Objectifs et choix techniques

L'objectif initial a donc été de créer une plateforme web de réservation d'équipements.

Il s'agissait donc de créer :

1. une API REST complète avec une architecture modèle repository service et contrôleur,
2. une application web permettant :
 - a. la gestion des équipements et de leurs dépendances (requis, recommandé, compatibles)
 - b. de la gestion des utilisateurs et de leur niveau d'autorisation,
 - c. la gestion des réservations.

J'ai choisi d'utiliser initialement la stack Java / Angular pour la solidité de celle-ci, utilisée de manière répandue en milieu professionnel et dans de multiples domaines critiques (santé, industrie,...). Cette stack était aussi l'opportunité pour moi de me challenger sur des langages de programmation inconnus (Java/TypeScript).

Durant la réalisation du projet, j'ai modifié mon choix de framework de Angular à React, la courbe d'apprentissage d'Angular étant trop importante au vu de l'avancement du projet. Couplé à TypeScript, React garde une rigueur dans le typage du code, ce qui réduit les cas d'erreur.

Pour ce projet j'utilise donc de multiples outils.

Pour le back-end Java, je me base sur **Spring**, un framework dédié à la conception d'applications Java, qui se détache en plusieurs modules, dont SpringBoot pour la configuration de projet Java, SpringDataJPA pour la gestion des entités, SpringWeb pour toutes les interactions avec le web, Spring Security pour renforcer la sécurité de l'application, et Spring Validation pour ajouter des règles de validations aux données (notamment des objets transférés de l'application web à l'api).

De plus, j'utilise les dépendances **Lombok** pour éviter le boilerplate (écriture de code inutile), **MapStruct** pour simplifier les conversions des entités en DTO (data transfer object) et inversement, et H2 pour la configuration de la base de données en local, ainsi que des frameworks de test (Web MVC test).

Pour ce qui est du front-end, j'ai choisi react ts comme framework (bibliothèque plus exactement) avec le langage TypeScript. Pour la gestion des styles j'utilise Tailwind et heroicons, la gestion des requêtes est en partie facilitée par tanstackQuery, react-date-picker et date-fns pour la gestion de dates,

Déroulement du projet

Planning

Mois	Janv		Février				Mars				Avril		
Numéro de semaine	1	2	3	4	5	6	7	8	9	10	11	12	13
Initialisation des différents projets													
Formation Java & Angular													
Création du modèle métier													
Création de la bdd et des entités + dtos													
Création des services, contrôleurs + tests													
Gestion de l'authentification côté back													
Création de la base du projet angular & premiers composants													
Gestion de l'authentification côté front													
Réalisation de la partie Utilisateur													
Réalisation de la partie Gestionnaire et Administrateur													
Intégration et tests													
Docker + CI/CD (optionnel)													

*planning prévisionnel à partir du point intermédiaire
(en rouge le planning réel)*

J'ai commencé par une phase d'apprentissage du langage Java, et angular. Voici une liste des ressources qui m'ont guidées tout du long de mon parcours d'apprentissage, puis durant la réalisation du projet :

- openclassrooms + projets indépendants pour maîtriser les différentes parties de spring (validation, security, data JPA)
- medium sur les conventions et certains tutoriels
- stackoverflow pour de la résolution de bugs

J'ai décidé d'implémenter en premier le back-end car j'avais déjà fait une maquette qui me permettait de me projeter dans la réalisation du front, donc en sachant les besoins du front, j'ai pu identifier les endpoints nécessaires au back et implémenter le modèle métier, et les autres couches de l'api.

Au fil de la réalisation du projet, je me suis rendu compte que le back-end prenait une part plus importante que prévue, notamment au niveau de la rédaction de la couche service et contrôleur, qui m'a fait redéfinir certains éléments du modèle. C'est donc pour cette raison que j'ai fait le choix de passer à React, un framework plus abordable pour lequel j'avais déjà acquis une légère expérience (projets d'école).

J'ai implémenté en continu les différentes fonctionnalités au niveau de l'application web, tout en adaptant le back selon des cas non couverts par les useCase mis en place précédemment.

Architecture de l'application

Back-end

L'api en back end est constituée de plusieurs couches :

- la couche modèle qui renferme les entités persistées en base de données et leurs relations
- la couche repository qui fait appel aux bases de données pour y effectuer des opérations de récupération, de modification, d'ajout, ou de suppression de données
- la couche service qui traite toute la partie métier et renvoie des données prêtes à être transférées sous forme de dto
- la couche contrôleur qui assigne un chemin d'accès aux méthodes du service

À côté de ceci sont stockés les DTO (data transfer objects), répertoriés selon leur usage (request pour des requêtes, response pour le renvoi au front, resume pour des

usages où il n'y a pas besoin de toutes les données d'une entité, ou pour éviter les appels circulaires), et dont les méthodes de conversion d'entité à dto et inversement sont décrites dans des mappers, qui permettent de faciliter la conversion sans écrire duplication de code (raison de l'usage de MapStruct)

On trouve aussi dans notre arborescence les fichiers exception qui sont utilisés par le service lorsqu'une erreur est rencontrée. Ces dernières sont récupérées par un Gestionnaire d'exception qui renvoie directement une réponse http au front-end. Cela facilite considérablement la gestion des contrôleurs qui n'ont plus besoin de gérer les exceptions levées dans le service, et permet un masquage des données critiques que l'on ne souhaite pas transmettre au front.

Les enums sont construits de pair avec un converter, qui permet via un code de ne pas perturber les entrées persistées avant une modification des valeurs de ce dernier.

Le fichier pom.xml renferme toutes les informations dédiées aux dépendances. Le fichier application.properties contient des informations de configuration de projet. Ce dernier a été suppléé par deux fichiers application-dev.properties et application-prod.properties pour pouvoir séparer les configurations selon l'environnement dans lequel l'application s'exécute.

Les tests sont renfermés dans un dossier test, regroupés par couche (service, controller, dto).

Concernant la sécurité, les fichiers liés à la mise en place de la sécurité sont répertoriés dans un fichier Configuration, où l'on rassemble la chaîne de filtres de sécurité SpringSecurityConfig (qui permet de déterminer les règles d'accès selon les routes appelées), le service de gestion d'authentification CustomUserDetailsService, et le filtre et service lié à la mise en place de JWT, un token sécurisé permettant de transférer de manière sécurisée des données à l'aide d'une signature.

En annexe se trouve une capture de la structure décrite ci-dessus pour le front-end et le back-end, ainsi que le graphique de classes UML.

Front-end

Le projet React possède une architecture dite en features, ce qui facilite la recherche d'éléments selon la fonctionnalité implémentée. On retrouve dans ce projet un dossier assets qui contient des images utilisées dans le projet, le dossier src contenant les principaux dossiers du projet. Ces dossiers sont construits de la manière suivante, on y trouve une structure standardisée :

- /api pour gérer les appels à l'api,
- /hooks pour accéder et traiter plus facilement les fonctions d'appel API
- /components , éléments d'UI trop complexe pour être intégrés à une page ou réutilisés plusieurs fois
- pages/, contient uniquement les pages complètes
- types/. rassemble les types utilisés

Cette structure se trouve donc dans le dossier App, dans chaque feature, et dans un dossier shared, utilisés pour les éléments utilisés en commun.

On retrouve aussi dans le fichier App toute la configuration du router grâce à **React Router**, framework qui permet à l'application de passer d'une SPA (single page application) à une application multi page. Sont définies alors des routes aux accès contrôlés, avec des pages de redirection en cas d'erreur ou bien de déconnexion.

Voici la liste des pages de l'application web

Pages dédiées aux utilisateurs :

- /home, la page d'accueil
- /aide, explique le fonctionnement de la plateforme
- /equipements, catalogue complet des équipements disponibles à la réservation
- /equipements/:id — La fiche détaillée d'un équipement avec ses exemplaires et ses labels
- /labels/:id — La liste des équipements regroupés sous un label donné

- /exemplaires/:id, fiche d'un exemplaire avec ses créneaux réservés passés et à venir
- /disponibilites, calendrier pour visualiser les disponibilités d'un ou plusieurs équipements sur le mois
- /reservations/creer, formulaire de création de réservation
- /reservations/mes-reservations , rassemble les réservations de l'utilisateur connecté, filtrables par statut
- /reservations/:id — Le détail d'une réservation avec ses sessions, son statut et son historique
- /profile, page de profil de l'utilisateur avec ses informations et les actions administrateur si applicable
- /profile/edit, formulaire de modification des informations personnelles

Pages admin

- /admin/users, gestion des utilisateurs de la plateforme.
- /admin/users/:id, vue du profil d'un utilisateur par l'administrateur.
- /admin/users/:id/edit, modification des informations d'un utilisateur.
- /admin/equipements, gestion des équipements
- /equipements/ajouter-equipement, formulaire de création d'un nouvel équipement.
- /admin/equipements/:id/modifier, formulaire de modification d'un équipement existant
- /admin/exemplaires, gestion des exemplaires
- /admin/labels, gestion des labels.
- /admin/reservations, vue globale de toutes les réservations avec calendrier et filtres par statut.

Pages authentication

- /login — La page de connexion à la plateforme
- /initialisation — La page d'activation d'un compte suite à une invitation

Description des éléments du projet

Authentification

Un utilisateur est ajouté nécessairement par un administrateur en renseignant son email, qui crée manuellement un compte avec un mot de passe d'invitation permettant à l'utilisateur d'initialiser son compte.

Le compte créé possède ainsi une date limite de validation que l'administrateur peut changer.

L'expiration du compte retire les droits d'accès à la plateforme.

La date d'expiration peut être modifiée par n'importe quel administrateur.

Chaque utilisateur peut modifier les champs de son profil.

En cas d'oubli, les réinitialisations de mot de passe doivent se faire via une demande aux administrateurs, qui appliqueront manuellement la réinitialisation et communiqueront un mot de passe temporaire généré automatiquement, afin que les utilisateurs puissent récupérer leur compte.



Connexion

Connectez-vous pour accéder à l'application

SE CONNECTER



Retour à la page Utilisateurs

Bernard Claire

BC

Bernard Claire
claire@test.com

Informations

FORMATION Master Biomécanique	RÔLE USER
STATUT ACTIF	DATE D'EXPIRATION 20/10/2026

MODIFIER LE PROFIL

Administration

Changer la date d'expiration

jj / mm / aaaa

METTRE À JOUR

RÉINITIALISER LE MOT DE PASSE

SUPPRIMER LE COMPTE

Page d'authentification et gestion des profils

Équipements

Les administrateurs ont la possibilité d'ajouter /modifier / supprimer des équipements.

La liste des équipements est disponible via la page catalogue pour les utilisateurs, et dans la page gestion des équipements pour les administrateurs.



catalogue des équipements

Relations entre équipements

Un équipement a la possibilité de posséder des relations avec d'autres équipements. Il existe 3 degrés d'association : requis, recommandé, compatible. Un équipement ne peut pas être réservé sans ses équipements requis.

Labels

Il est possible d'assigner des labels à des équipements.

L'ajout / modification / suppression des labels est accessible aux administrateurs via le centre de contrôle.

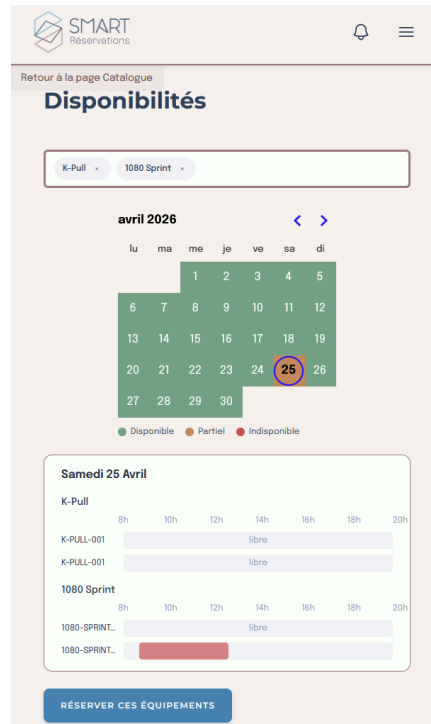
La recherche d'équipements dans le catalogue peut être regroupée par labels, et chaque label possède sa propre page, permettant d'avoir un aperçu des équipements assignés à ce dernier.

Exemplaires

Chaque équipement possède un ou plusieurs exemplaires, donc l'ajout, la modification, et la suppression sont accessibles aux administrateurs via le centre de contrôle. Chaque équipement possède sa propre page, avec les emprunts auxquels il est lié. Un exemplaire possède plusieurs états : disponible

Disponibilités

Les utilisateurs ont la possibilité de consulter la disponibilité d'un équipement en fonction d'une date via une page dédiée. Cette page "Voir les disponibilités" permet de sélectionner plusieurs équipements, et de voir pour cette sélection via une vue calendrier les disponibilités selon la journée. Les détails des disponibilités d'une journée sont accessibles en cliquant sur un des jours du calendrier, révélant les créneaux horaires des différents exemplaires de chaque équipement sélectionné.



vue de la page disponibilités

Réservations, sessions et emprunts

Chaque utilisateur a la possibilité de créer une réservation. Une demande réservation contient un ou plusieurs sessions, avec pour chaque session une liste d'équipements. Afin de faciliter la gestion des réservations, la sélection des équipements a été contrainte à rester la même pour chaque session réservée, malgré qu'il soit possible d'en faire la personnalisation si besoin dans le futur.

A la création d'une nouvelle réservation, cette dernière est mise en attente, et doit être alors validée ou refusée par les administrateurs. A la validation d'une réservation, des emprunts sont créés en fonction des créneaux réservés. Un emprunt est associé à un seul et unique exemplaire réservé.

Poursuite du projet

Le projet malgré son avancement peut être grandement amélioré. En effet, l'aspect UX, malgré la maquette, a des points à améliorer, notamment au niveau de la réalisation d'une réservation.

Les notifications aux utilisateurs seront aussi à intégrer dans la plateforme, de manière interne, et de manière externe

La cohérence du design est aussi à revoir, et des éléments du code mériteraient une refactorisation pour avoir un code plus accessible et compréhensible, notamment pour le front.

La poursuite du projet va consister en sa dockerisation et son déploiement via render pour le back et netlify pour le front (la configuration du projet étant quasiment prête pour le déploiement à l'exception de l'intégration d'un gestionnaire de migrations pour les bases de données), en la complétion de tests, en son renforcement des fonctionnalités pour augmenter la robustesse, et l'intégration de nouvelles méthodes de connexion.

Bilan personnel

Malgré l'envergure du projet, ce dernier m'a appris beaucoup de choses, notamment du côté java et spring, et notamment sur les pratiques les plus appropriées pour mener un projet qui soit le plus professionnel possible, au niveau du front-end comme pour le back-end.

Cependant, je ne peux que souligner le manque de rigueur dans les commits qui ne permettra pas une bonne traçabilité de la construction du projet.

J'aurais peut-être gagné à effectuer une approche en MVP, en ayant une version simplifiée mais finie pour le front-end et le back-end.

Aussi, j'aurais souhaiter utiliser une approche dirigée par les tests, cependant j'ai appris durant le projet à les réaliser, et ne m'a pas permis dès le départ du projet à adopter le réflexe de réaliser un test simple afin de fixer un comportement qui doit se produire ou non, puis implémenter la feature et corriger/refactoriser ensuite.

C'est une façon de travailler que j'aurais aimé explorer.

Conclusion

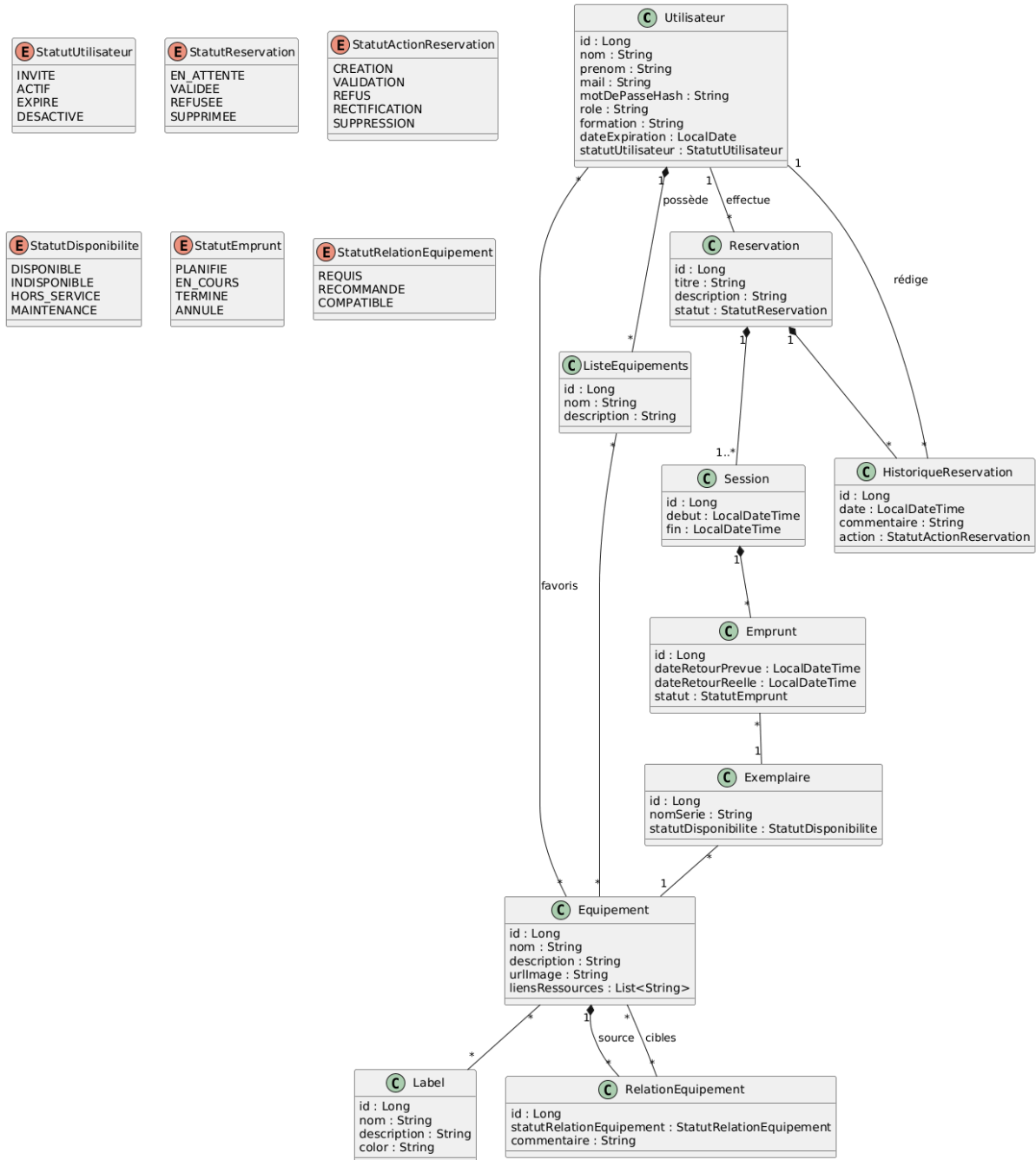
Le projet SMART Réservations a abouti à la réalisation d'une plateforme web fonctionnelle répondant aux objectifs initiaux : une API REST Spring Boot couvrant la gestion des équipements, des exemplaires, des utilisateurs et des réservations, couplée à une plateforme web React TypeScript permettant aux utilisateurs de consulter le catalogue, de visualiser les disponibilités et de soumettre des demandes de réservation, et aux administrateurs de les traiter.

Le périmètre fonctionnel livré comprend l'ensemble des parcours utilisateur identifiés en début de projet — authentification par invitation, gestion des relations entre équipements, validation des réservations, et constitue une base exploitable pour un déploiement en conditions réelles au sein du bâtiment SMART.

Les principaux axes d'amélioration identifiés portent donc sur la cohérence UX de la plateforme, la couverture de tests du frontend, et la mise en place d'une chaîne de déploiement (Docker, CI/CD) pour faciliter la mise en production. Ces éléments, bien que n'ayant pas pu être mis en place dans les temps, ont été documentés et pourront être pris en charge dans le cadre d'une poursuite du projet.

Annexe

Diagramme de classes du back-end



Structure du Front-end (à gauche) et du back-end (à droite)

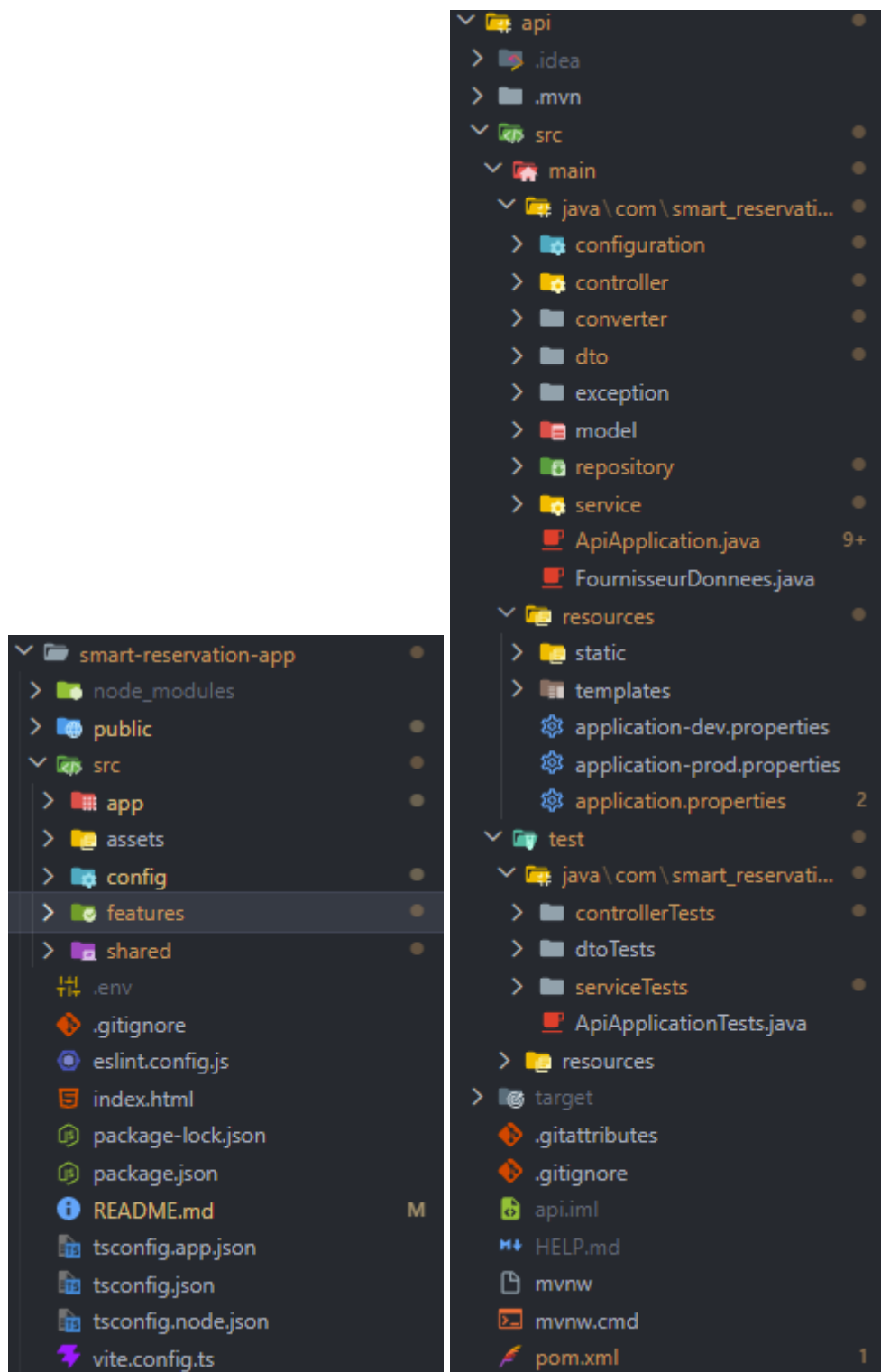


Diagramme d'organisation des pages

